PROJECT PROPOSAL

SentinelLLM

A Multimodal Agentic Framework for Endpoint Compromise Detection, Unauthorized File Analysis, and Safe Automated Remediation

Project Vision
Build an evidence-driven Windows security assistant that can detect suspicious execution chains, credential/session-stealing behavior, unauthorized files, persistence mechanisms, and visual social-engineering attacks - then explain the incident and perform policy-controlled quarantine without allowing an LLM to delete files autonomously.

Prepared by
Md Mizanur Rahman
Research & Development Proposal
September 2026

1. Executive Summary

SentinelLLM is a hybrid endpoint-security research project designed to answer four practical questions after a suspected compromise: Is the computer currently compromised? What suspicious or unauthorized artifacts appeared? What attack chain produced them? What remediation can be performed safely without destroying legitimate files? The system combines deterministic security rules, Windows telemetry, file integrity and reputation analysis, static and behavioral machine learning, sandbox evidence, network intelligence, a vision-language model (VLM) for visual social-engineering detection, and an evidence-grounded large language model (LLM) for incident reconstruction and explanation.

The core principle is that generative AI is not the source of truth. Security events, signatures, hashes, process ancestry, file access, network behavior, and antivirus/sandbox results form the evidence layer. The LLM reasons over structured evidence, while a separate policy engine controls high-impact actions such as process termination, network isolation, quarantine, persistence removal, and final deletion.

Primary research hypothesis
Combining behavioral endpoint telemetry with multimodal social-engineering detection and evidence-grounded LLM reasoning can improve attack-chain detection and analyst comprehension while maintaining a low benign-quarantine rate.

2. Problem Statement

Modern credential-stealing and session-hijacking attacks often use short-lived scripts, encrypted archives, randomly named executables, browser-session theft, and social-engineering pages. A payload may execute for only seconds, exfiltrate valuable authentication material, and then disappear or be removed by antivirus. Traditional signature-based detection may identify an individual payload but often does not reconstruct the full sequence from deceptive webpage to script execution, credential-store access, outbound communication, and later account misuse.

A second problem is safe remediation. Automatically deleting every unfamiliar executable is unacceptable because legitimate software updates, developer tools, drivers, and user-created binaries can appear new or unsigned. SentinelLLM therefore treats deletion as the final stage of a controlled response pipeline rather than the initial reaction.

3. Aim and Objectives

Aim: Develop and evaluate a local-first intelligent endpoint security framework capable of detecting, explaining, and safely responding to suspicious Windows activity and unauthorized files using multiple independent evidence sources.

- Detect suspicious process chains, scripts, file creation, persistence, browser-store access, and anomalous network behavior in near real time.
- Identify newly introduced or modified executable/script artifacts using baselines, hashes, digital signatures, path context, and reputation.
- Detect fake CAPTCHA, ClickFix-style verification, fake login, fake update, and deceptive download interfaces using a VLM.
- Use traditional ML to score static and behavioral features before invoking an LLM.
- Use an LLM to correlate heterogeneous evidence into an explainable attack timeline and threat hypothesis.
- Implement a deterministic risk-fusion and policy engine that can allow, alert, suspend, isolate, quarantine, or recommend removal.

- Correlate endpoint incidents with later account-security events where supported by service APIs or exported security logs.
- Evaluate detection quality, false positives, latency, resource usage, explanation grounding, and remediation safety.

4. Scope and Threat Model

In Scope	Examples
Execution attacks	PowerShell, CMD, script interpreters, downloaded executables, LOLBins
Credential/session theft	Suspicious access to browser cookie/login/session stores; token-stealing behavior
Persistence	Run keys, scheduled tasks, services, startup folders, WMI persistence
Unauthorized files	New or modified EXE/DLL/PS1/BAT/JS/VBS and high-risk archives
Network behavior	Rare domains, suspicious process-to-domain relationships, exfiltration indicators
Visual deception	Fake CAPTCHA/Cloudflare verification, fake login/update/download prompts
Account correlation	New-login/device/security/billing events when authorized data is available

Out of scope for the first prototype: kernel-level EDR replacement, destructive malware detonation on the host, autonomous deletion based only on LLM output, collection of raw user passwords/cookies for analysis, and offensive exploitation of third-party systems.

5. Proposed System Architecture

Layer	Primary Components	Output
1. Sensors	Windows Event Logs, ETW/Sysmon-style telemetry, Defender, filesystem watcher, registry/tasks/services, network metadata, browser metadata	Normalized security events
2. Evidence Enrichment	SHA-256, Authenticode, YARA, reputation, PE features, path/first-seen context	Enriched artifact/event records
3. Detection	Rules + anomaly scoring + static ML + behavioral ML + optional sandbox	Independent detector scores
4. Multimodal AI	VLM for deceptive UI; LLM for evidence correlation and incident explanation	Threat hypothesis + grounded rationale

Layer	Primary Components	Output
5. Risk Fusion	Weighted/calibrated evidence aggregation and confidence thresholds	Risk score and response class
6. Policy & Response	Allow, observe, alert, suspend, block network, quarantine, remove persistence, delete after confirmation	Auditable remediation action

Safety boundary
The LLM and VLM may recommend actions, but they do not directly execute destructive commands. A deterministic policy engine validates evidence thresholds and action permissions.

6. Core Technical Modules

6.1 Endpoint Telemetry Collector

A lightweight Windows agent will collect process creation and ancestry, command lines, file events, signatures, Defender alerts, registry persistence changes, scheduled tasks, services, WMI subscriptions, network connections, DNS metadata, and selected browser-security metadata. Events will be normalized into a local schema with timestamps and process identifiers so that attack chains can be reconstructed.

6.2 Baseline and Unauthorized File Detector

The system will maintain a trusted baseline and record first-seen artifacts. Each executable or script will be described by SHA-256, path, timestamps, signer/publisher, parent process, command line, source context when available, network destinations, persistence behavior, and prior prevalence. A file is not considered malicious merely because it is new or unsigned; contextual evidence determines risk.

6.3 Behavioral Rule Engine

High-signal rules will capture suspicious chains such as hidden PowerShell downloading code, ExecutionPolicy bypass, script-to-archive-to-executable staging, execution from temporary directories, unsigned processes reading browser credential stores, new executables creating persistence, and suspicious processes immediately contacting rare external domains.

6.4 Static and Behavioral Machine Learning

Static PE/script features and event-sequence features will be evaluated using interpretable baselines and stronger tabular models. Candidate models include Logistic Regression, Random Forest, XGBoost/LightGBM, and a compact neural model. The ML layer reduces the number of events requiring expensive LLM reasoning.

6.5 VLM Social-Engineering Detector

A compact VLM will analyze screenshots only when a high-risk browser interaction is detected. Target classes include fake verification/CAPTCHA pages, ClickFix instructions, fake browser updates, deceptive download buttons, fake antivirus warnings, and credential-phishing interfaces. Visual evidence will be fused with clipboard and execution events rather than used alone.

6.6 Evidence-Grounded LLM Analyst

The LLM receives structured event summaries rather than unrestricted filesystem access. It reconstructs the likely attack chain, identifies supporting and contradicting evidence, maps behaviors to security concepts, estimates confidence, and produces an analyst-readable explanation. Outputs must reference event IDs so hallucinated claims can be detected.

6.7 Safe Remediation Engine

The preferred response sequence is detect -> suspend -> restrict network -> quarantine -> preserve evidence -> verify -> remove. Permanent deletion is reserved for policy-approved cases with strong corroboration, such as agreement among antivirus/YARA, behavioral evidence, reputation, and sandbox analysis. Every action is logged and reversible where possible.

7. Example Attack-Chain Detection

Observed Event	Evidence Interpretation	Risk
Browser displays verification page requesting command execution	Possible social-engineering/ClickFix pattern	Medium
PowerShell launches hidden with ExecutionPolicy Bypass	High-risk execution characteristic	High
PowerShell downloads remote script	Remote payload staging	High
Script extracts password-protected archive to Temp	Evasion/staging behavior	High
Random unsigned executable launches from Temp	Suspicious payload execution	High
Executable accesses browser session databases	Possible credential/session theft	Critical
Executable contacts rare external domain	Possible command-and-control/exfiltration	Critical
AV/YARA also detects payload	Independent corroboration	Critical

The framework should identify the chain as a single incident rather than producing isolated alerts. This correlation is a central research contribution.

8. Risk-Fusion Strategy

The first prototype will use transparent weighted evidence, followed by calibration on validation data. Example weights are illustrative only and will not be treated as final scientific parameters.

Signal	Illustrative Weight
Unsigned executable launched from Temp	+15
Hidden PowerShell	+15
ExecutionPolicy Bypass	+10

Signal	Illustrative Weight
Remote script download	+15
Password-protected archive staging	+10
Browser credential-store access by unknown process	+20
Rare outbound destination	+15
Independent AV/YARA detection	+40
Valid trusted publisher signature	-30
Known-good hash / allowlist	-50

Risk classes: 0-19 Normal; 20-39 Observe; 40-59 Suspicious; 60-79 High; 80-100 Critical. Thresholds will be tuned using validation data with explicit attention to benign quarantine rate.

9. Data and Experimental Methodology

The research dataset will combine public benign/malicious executable sources, PowerShell/script attack corpora, controlled Windows attack simulations, sandbox traces, deceptive webpage screenshots, and benign daily-use workloads. Personal incident artifacts should be treated as motivating case evidence, not as the sole experimental dataset. All malware execution must occur in isolated, disposable virtual machines with no access to personal credentials or production networks.

Experiment	Purpose
Benign workload baseline	Measure false positives during normal browsing, development, updates, office work, and software installation
Controlled staged-download attacks	Test script -> archive -> executable correlation
Credential-access simulations	Test browser-store access detection without collecting real secrets
Persistence simulations	Evaluate Run keys, scheduled tasks, services, and WMI detection
Visual deception set	Evaluate VLM on fake verification/login/update/download pages
Ablation study	Measure contribution of rules, ML, LLM, VLM, and threat intelligence
Adversarial robustness	Test obfuscation, renamed files, trusted-looking filenames, and noisy benign activity

10. Evaluation Metrics

Dimension	Metrics
Detection	Precision, Recall, F1, ROC-AUC/PR-AUC, false-positive rate
Operational	Detection latency, CPU, RAM, storage overhead, events/second
Security coverage	Credential-theft, persistence, staged-execution, ClickFix, malicious-file quarantine detection rates
LLM quality	Incident classification accuracy, evidence-grounding rate, unsupported-claim/hallucination rate
VLM quality	Precision/Recall/F1 for visual social-engineering classes
Remediation safety	Malicious quarantine rate, benign quarantine rate, successful rollback rate

11. Ablation Plan

Configuration	Purpose
Rules only	Deterministic baseline
ML only	Learned detector baseline
LLM only	Demonstrate limitations of generative-only security
Rules + ML	Hybrid non-generative baseline
Rules + ML + LLM	Measure reasoning/correlation benefit
Rules + ML + LLM + VLM	Measure visual social-engineering benefit
Full system + reputation/threat intelligence	End-to-end configuration

12. Technology Stack

Area	Candidate Technologies
Endpoint prototype	Python; Windows Event Log APIs; WMI/CIM; optional ETW/Sysmon integration
Storage	SQLite/DuckDB for local prototype; structured JSON event schema
File analysis	SHA-256, Authenticode, PE parsing, YARA
ML	scikit-learn, XGBoost/LightGBM, PyTorch where needed

Area	Candidate Technologies
LLM	Local 1.5B-4B instruct model or secured API; structured JSON prompting
VLM	Compact Qwen-VL/Phi-Vision-class model depending on compute
Sandbox	Disposable Windows VM; optional CAPE/Cuckoo-style integration
Threat intelligence	Reputation feeds/APIs subject to licensing and privacy constraints
Dashboard	FastAPI backend + lightweight web UI or desktop dashboard

13. Development Roadmap

Phase	Weeks	Key Deliverable
1. Requirements & threat model	1-2	Event schema, attack taxonomy, safety policy
2. Endpoint telemetry	3-4	Process/file/network/persistence collector
3. Baseline + rule engine	5-6	First-seen inventory and deterministic detections
4. Static/behavioral ML	7-8	Trained baselines and calibrated risk scores
5. LLM forensic analyst	9-10	Evidence-grounded incident reconstruction
6. VLM browser defense	11	Visual deception detector and event fusion
7. Safe response engine	12	Suspend/isolate/quarantine workflow with audit log
8. Integrated evaluation	13-14	Attack simulations, benign tests, ablations
9. Optimization	15	Latency/resource/false-positive tuning
10. Paper & release package	16	Paper draft, reproducible experiments, demo

14. Expected Research Contributions

- A hybrid endpoint framework that separates evidence collection, detection, reasoning, and remediation authority.

- Attack-chain reconstruction that correlates visual social engineering with subsequent endpoint execution and credential-access behavior.
- An evidence-grounded LLM analyst whose claims are traceable to structured event IDs.
- A privacy-aware browser-session protection mechanism based on access metadata rather than extracting user secrets.
- A calibrated safe-remediation policy that evaluates benign quarantine rate as a first-class metric.
- A reproducible benchmark covering benign activity, staged malware execution, persistence, credential-access simulations, and deceptive web interfaces.

15. Safety, Privacy, and Ethical Controls

- No collection or storage of raw passwords, cookie values, authentication tokens, CVVs, or full payment-card numbers.
- All malware detonation occurs in isolated disposable VMs with synthetic accounts and no personal data.
- LLM/VLM outputs are advisory evidence; destructive actions require deterministic policy approval.
- Quarantine is preferred over permanent deletion, with hashes and metadata preserved for audit and rollback.
- Telemetry is minimized, stored locally by default, and protected by access controls.
- Experiments use authorized systems only and do not attempt unauthorized access to external services.

16. Key Risks and Mitigations

Risk	Mitigation
False positive deletes legitimate software	No LLM-controlled deletion; quarantine first; multi-signal confirmation; rollback
LLM hallucination	Require event-ID grounding; schema-constrained outputs; unsupported-claim metric
VLM misreads benign verification pages	Fuse visual result with clipboard/process/network evidence
High endpoint overhead	Event filtering, asynchronous enrichment, caching, selective LLM/VLM invocation
Dataset leakage or unrealistic benchmark	Separate train/test families; controlled time-based splits; real benign workloads
Privacy exposure	Metadata-first collection; redact secrets; local processing; short retention

17. Deliverables

- Windows endpoint telemetry collector and normalized event schema.
- Baseline inventory and unauthorized-file detection module.
- Rule engine plus static and behavioral ML models.
- VLM-based visual social-engineering detector.
- Evidence-grounded LLM incident analyst.
- Risk-fusion and policy-controlled quarantine/remediation engine.
- Local dashboard showing attack timeline, evidence, risk, and response actions.
- Controlled evaluation dataset/scripts, ablation results, and reproducibility package.

- Conference/journal-ready research manuscript and demonstration video.

18. Success Criteria

The prototype will be considered successful if it can detect representative staged credential-stealing and persistence scenarios with strong recall while maintaining a low false-positive and benign-quarantine rate; reconstruct the major attack stages from structured evidence; identify deceptive browser interfaces with measurable VLM benefit; operate with acceptable endpoint overhead; and ensure that destructive remediation never occurs solely because an LLM or VLM labels an artifact malicious.

19. Proposed Final System Output

Example incident report
CRITICAL - Possible credential/session stealer

Attack chain: browser deception -> hidden PowerShell -> remote script -> encrypted archive -> unsigned Temp executable -> browser-store access -> rare outbound connection.

Evidence: process ancestry, command line, file hash/signature, credential-store access, network destination, independent AV/YARA result.

Response: process suspended; network restricted; payload quarantined; persistence checked; forensic timeline preserved.

User action: revoke affected account sessions from a trusted device and review account-security events.

20. Conclusion

SentinelLLM is proposed as an explainable, multimodal, defense-in-depth endpoint security framework rather than an LLM-based antivirus. Its novelty comes from joining visual social-engineering detection, endpoint behavioral telemetry, unauthorized-file analysis, machine-learning risk scoring, evidence-grounded LLM reasoning, and policy-controlled remediation into one auditable workflow. The project is designed to produce both a practical prototype and a rigorous research evaluation suitable for publication.